

Module 1 — Core Mastery

Lesson 1.3

Slash commands you'll actually use

A short list of built-in slash commands you'll touch every day, and what they really do.

Mastering Claude Code — An E-Course for Power Users

Why it matters

`/help` lists everything. Most of it you'll use rarely. This lesson is the 20% you'll use 80% of the time, with the gotchas.

The daily set

Context & session

Command	What it does	When
<code>/help</code>	Lists all available slash commands and skills	When in doubt
<code>/clear</code>	Wipes in-session conversation context	Switching tasks mid-session
<code>/compact</code>	Forces context compaction now	Long session, feeling sluggish
<code>/resume</code>	Picker for past sessions in this dir	Coming back the next day
<code>/exit</code>	Quit the session	End of work

`/clear` vs. exit-and-restart: `/clear` keeps the session record, your settings, your loaded MCP servers. Restarting reloads everything from disk. Prefer `/clear` for "new task, same setup."

Configuration

Command	What it does
<code>/model</code>	Show or change the current model
<code>/config</code>	Open the interactive settings UI
<code>/agents</code>	List/manage subagents available in this session
<code>/hooks</code>	List/manage hooks
<code>/permissions</code>	View and edit the permissions list

Project bootstrapping

Command	What it does
<code>/init</code>	Generate or refresh <code>CLAUDE.md</code> for the current repo
<code>/login</code> / <code>/logout</code>	Subscription auth management

`/init` is worth running once per repo on day one. It reads the repo, drafts a `CLAUDE.md`, and gives you a base to edit.

Built-in skills (action commands)

These are higher-level workflows shipped as skills. They invoke a specialized routine, not just a prompt:

Command	What it does
<code>/review</code>	Reviews the current pull request or pending changes
<code>/security-review</code>	Security-focused review of pending changes
<code>/verify</code>	Runs the app and exercises a change to confirm it works
<code>/simplify</code>	Reviews recent changes for simpler alternatives

Skills are auto-discovered; what's available depends on your installed version and any plugin packs. Run `/help` to see what's live for you.

Custom slash commands

Beyond built-ins, you can ship your own. Drop a markdown file in `.claude/commands/` (project) or `~/.claude/commands/` (user-global). The body is the prompt; `$ARGUMENTS` is replaced with whatever the user types after the command name.

We cover this in detail in 3.4 Custom slash commands. Demo project: `examples/01-custom-slash-command`.

Plugin-namespaced commands

Plugins (and some MCP servers) expose commands under a namespace, e.g. `/myplugin:do-thing`. Use the fully qualified form when invoking. Run `/help` to see the namespaced list.

Try it

- 1 `/help` — read through everything available in your install. Note the ones you didn't know.
- 2 `/model` — see which model you're on. Switch to a different one and back.
- 3 `/init` — generate a `CLAUDE.md` in a fresh project. Skim what it produced.
- 4 `/clear`, then ask for the same task you were on. Note that the session no longer remembers context.

Gotchas

- **Don't type slash commands inside string arguments to tools.** They only work as standalone messages.
- ``/compact`` is not `/clear`. Compact summarizes-and-keeps; clear deletes. Compact is for "I want to keep going on this task with less baggage." Clear is for "different task."
- ``/init`` overwrites ``CLAUDE.md`` if it exists (after confirming). If you've hand-tuned yours, commit before running.
- **Skills are not slash commands.** Some skills are auto-invoked by Claude based on description matching. Slash form (`/skill-name`) forces invocation.

Further reading

- 3.4 Custom slash commands
- 1.5 Context management — when to compact vs. clear
- Official slash commands docs: <https://docs.claude.com/en/docs/claude-code/slash-commands>

Next

→ 1.4 Plan mode